AI_HW2

1、简答题

1.1

K折交叉验证可以避免数据浪费，且可以对抗"lucky date"类似的情况发生；K并不是越大越好，K越大复杂度越高。

1.2

基函数是将原始输入“升维”或“映射”到另一个空间，使得非线性问题在新空间中可以被线性模型解决；基函数越复杂，学习的效果不一定会越好，可能会导致过拟合。

1.3

bootstrap(bagging), 随机特征选择

1.4

Actor-Critic 将 J 换为了 Q，Q 是一个平均值，变化不大，因此方差较小。

1.5

神经网络是一个高非线性的函数逼近器，并不满足早期 Q-learning 理论中对逼近器（如线性逼近）的限制条件，还有 Q 的目标不是静态的，而是“朝着自己移动的目标追赶”，这很容易导致梯度爆炸或震荡，再加上强化学习的数据是时序相关的，不像监督学习是 i.i.d（独立同分布）的，训练过程中的状态分布会随策略改变而不断变化，也会影响学习稳定性。

使用基于策略的方法也不能保证收敛到全局最优策略。

2、ID3 算法的次优性

2.1

第一层：

对于x第一维：

$-\frac{2}{3}\log\frac{2}{3}-\frac{1}{3}\log\frac{1}{3}$

$-\frac{1}{1}\log\frac{1}{1}=0$

对于x第二维：

$-\frac{1}{2}\log\frac{1}{2}-\frac{1}{2}\log\frac{1}{2}=1$

$-\frac{1}{2}\log\frac{1}{2}-\frac{1}{2}\log\frac{1}{2}=1$

对于x第三维：

$-\frac{1}{2}\log\frac{1}{2}-\frac{1}{2}\log\frac{1}{2}=1$

$-\frac{1}{2}\log\frac{1}{2}-\frac{1}{2}\log\frac{1}{2}=1$

显然第一层以x第一维来分类。

对于第二层：

对于x第二维：

$-\frac{1}{2}\log\frac{1}{2}-\frac{1}{2}\log\frac{1}{2}=1$

$-1\log 1=0$

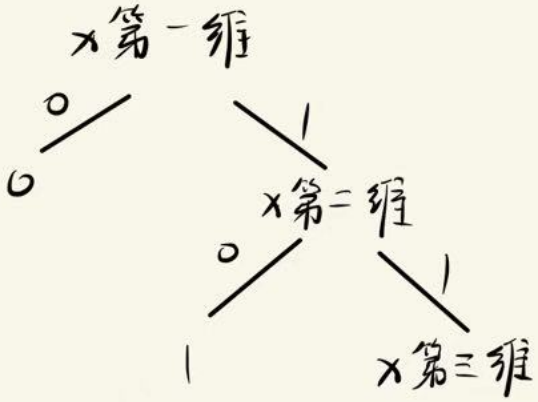
对于x第三维：

$-1\log 1=0$

$-\frac{1}{2}\log\frac{1}{2}-\frac{1}{2}\log\frac{1}{2}=1$

于是第二层在取.

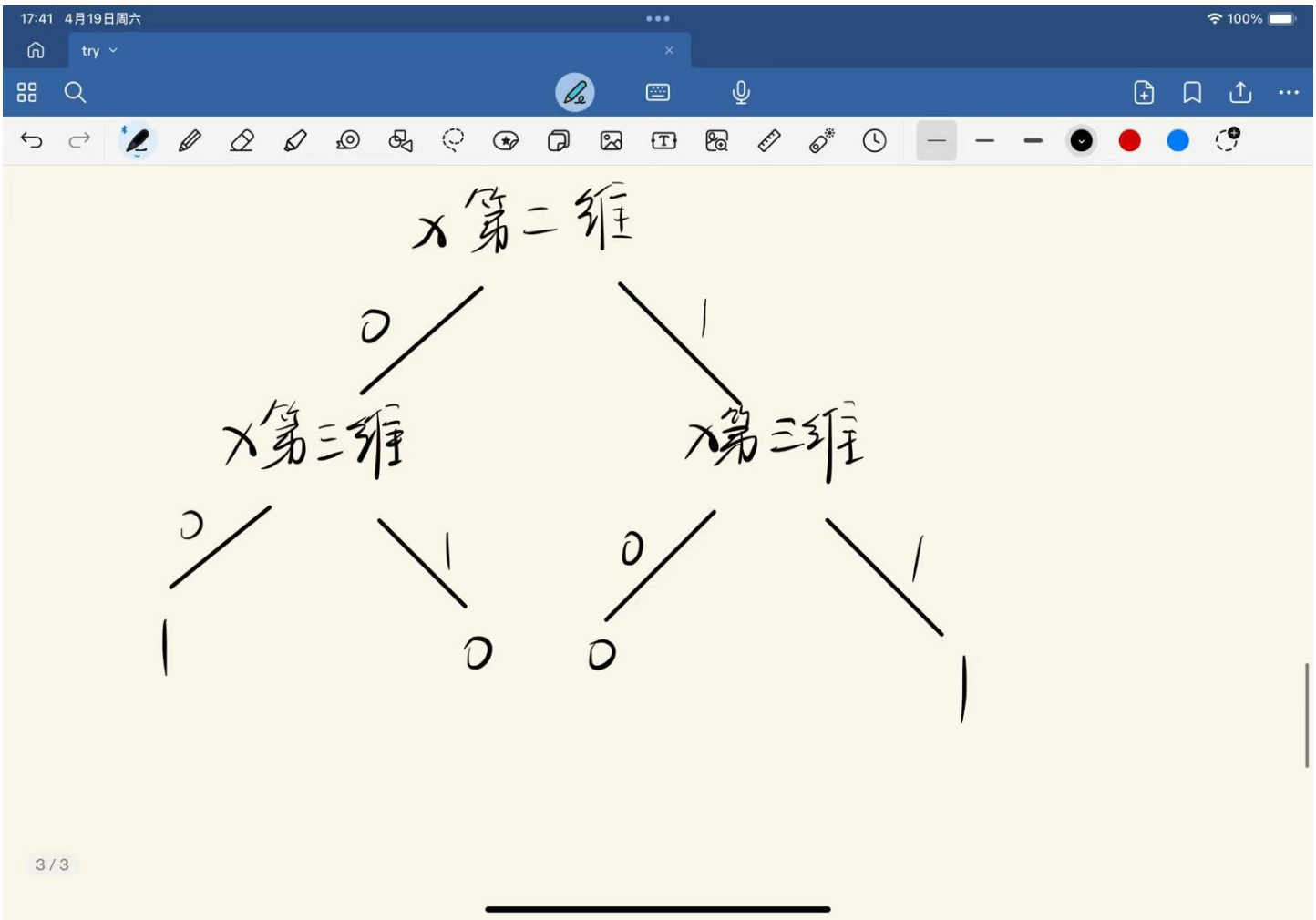
如下



以上是深度不超过2的决策树的构建过程。

当整个决策树构建完成后，其深度为3，但是如果整棵树深度不超过2的话，在用 x 的第二维分类时，如果为1，则结果必须为0，这样就会有一个样本被分类错误，因此训练误差至少为 1/4。

2.2



3、线性模型与 AlphaGoZero

3.1

令 $t = XW_v + b_v/B$

$$\frac{\partial L}{\partial t} = \frac{1}{B}\frac{\partial (z-v)^T(z-v)}{\partial t} = \frac{1}{B}\frac{\partial (z-\tanh t)^T(z-\tanh t)}{\partial t}$$

$$= \frac{1}{B}\frac{\partial (z-\tanh t)^T(z-\tanh t)}{\partial \tanh t}\cdot\frac{\partial \tanh t}{\partial t}$$

令 $\tanh t = S$

则 $$\frac{\partial L}{\partial t} = \frac{1}{B}\frac{\partial (z-S)^T(z-S)}{\partial S}\cdot\frac{\partial S}{\partial t}$$

$$= \frac{2}{B}(\tanh t - z)\cdot(1-\tanh^2 t) = \frac{2}{B}(v-z)\cdot(1-v^2)$$

$$\Longrightarrow \frac{\partial L}{\partial W_v} = \frac{\partial L}{\partial t}\cdot\frac{\partial t}{\partial W_v} = 2X^T(v-z)(1-v^2)/B$$

$$\frac{\partial L}{\partial b_v} = \frac{\partial L}{\partial t}\cdot\frac{\partial t}{\partial b_v} = 2_B^T(v-z)(1-v^2)/B$$

3.2

```
C:\Users\lenovo\.conda\envs\pytorch_cpu\python.exe -m model.linear_model
[Gradient of w_v] shape:(247,)      Mean Error: 0.000001, Max Error: 0.000001
[Gradient of b_v] shape:(1,)            Mean Error: 0.000000, Max Error: 0.000000
[Gradient of w_pi] shape:(247, 49)      Mean Error: 0.000000, Max Error: 0.000000
[Gradient of b_pi] shape:(49,)      Mean Error: 0.000000, Max Error: 0.000000

Process finished with exit code 0
```

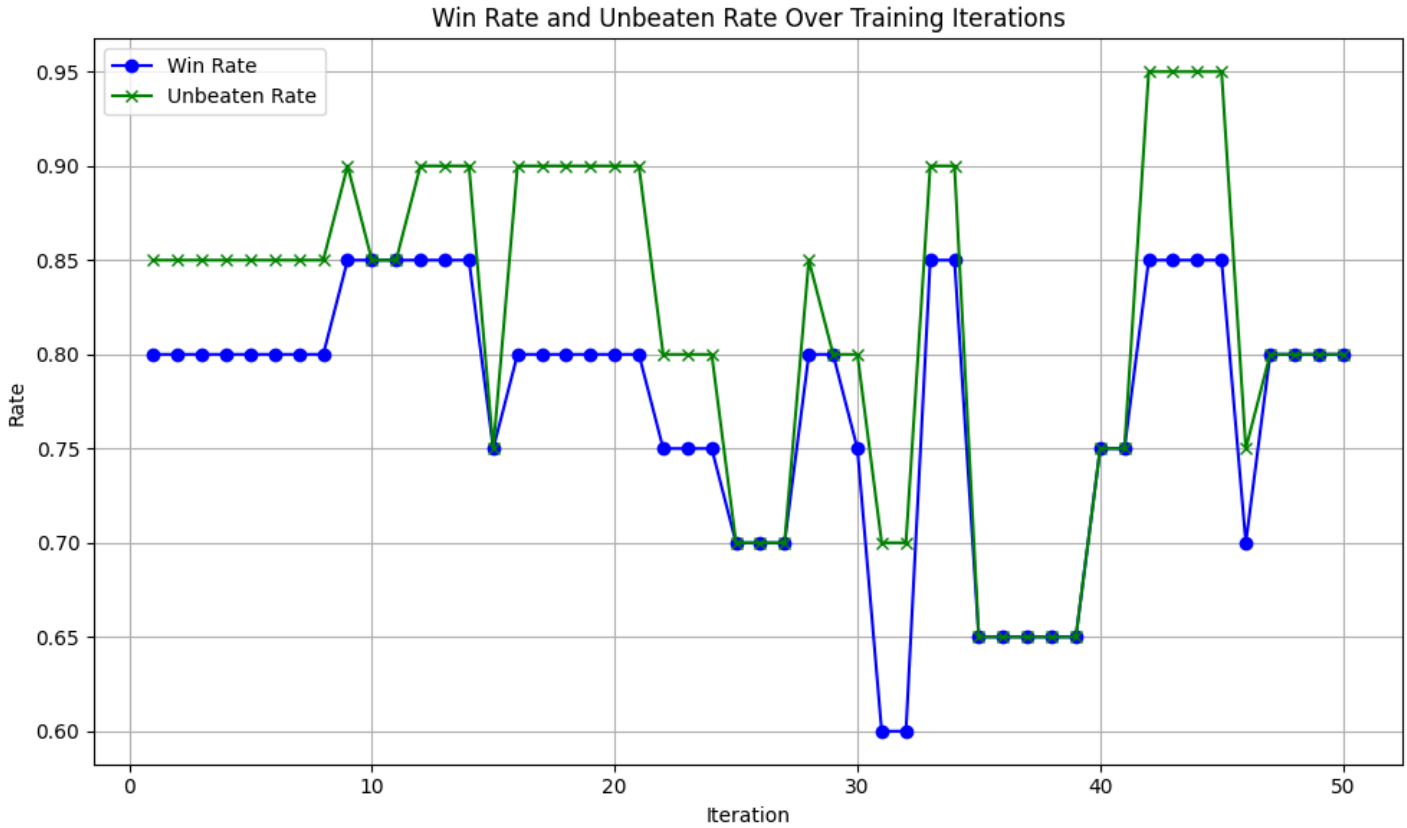
3.3

在附件中

3.4

参数选择：

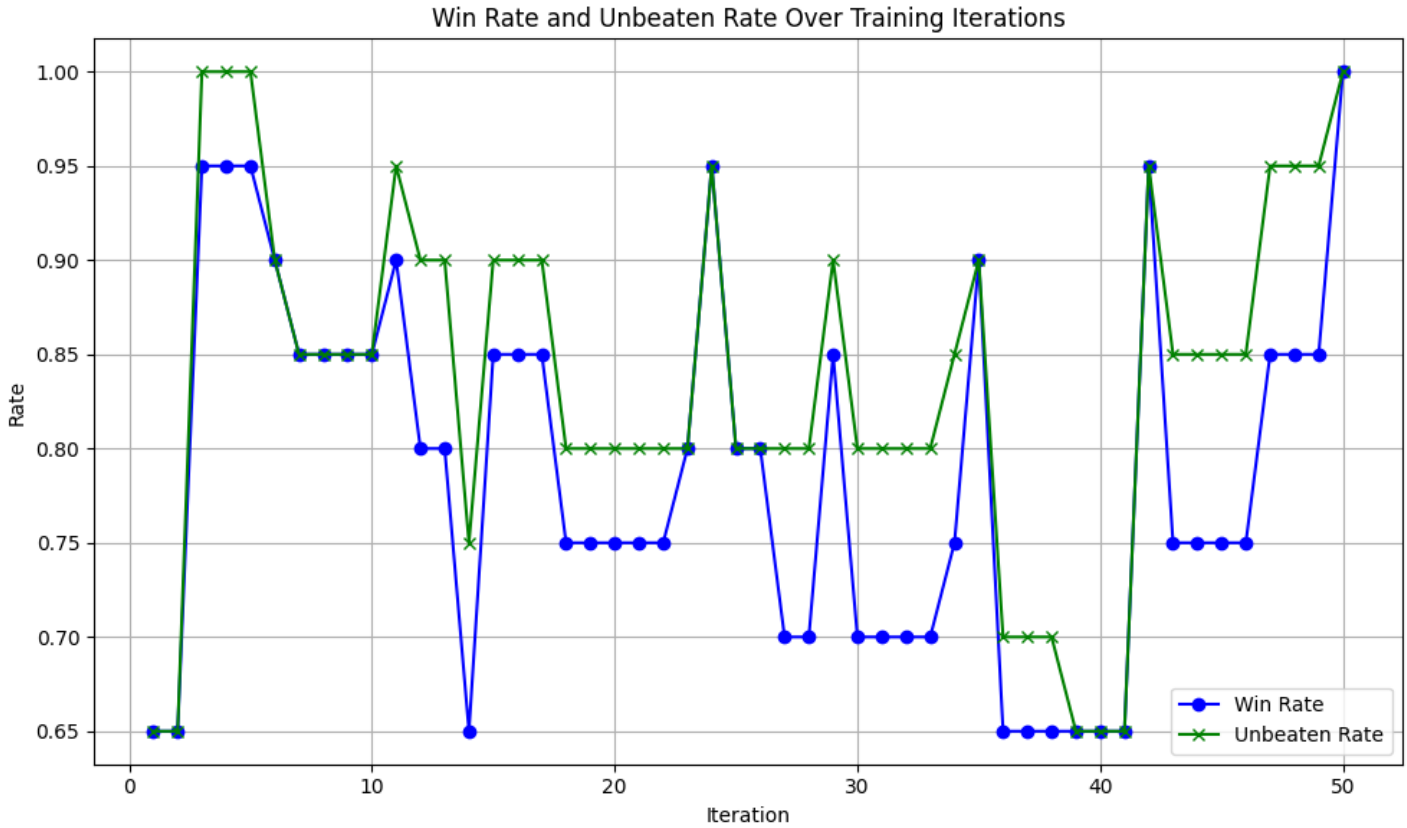
```
config = AlphaZeroConfig(
    n_train_iter=50,
    n_match_train=20,
    n_match_update=20,
    n_match_eval=20,
    max_queue_length=160000,
    update_threshold=0.501,
    n_search=240, # default 240, now is 500
    temperature=1.0,
    C=1.0,
    checkpoint_path="checkpoint/linear_7x7_exfeat_norm"
)
```



3.5

对 n_search 进行了修改，参数选择：

```
config = AlphaZeroConfig(
    n_train_iter=50,
    n_match_train=20,
    n_match_update=20,
    n_match_eval=20,
    max_queue_length=160000,
    update_threshold=0.501,
    n_search=500, # default 240, now is 500
    temperature=1.0,
    C=1.0,
    checkpoint_path="checkpoint/linear_7x7_exfeat_norm"
)
```



n_search 如何影响：

n_search 指的是每次进行决策时，模型执行MCTS搜索的次数。增加搜索次数通常可以提高模型的表现，但也会增加计算时间。

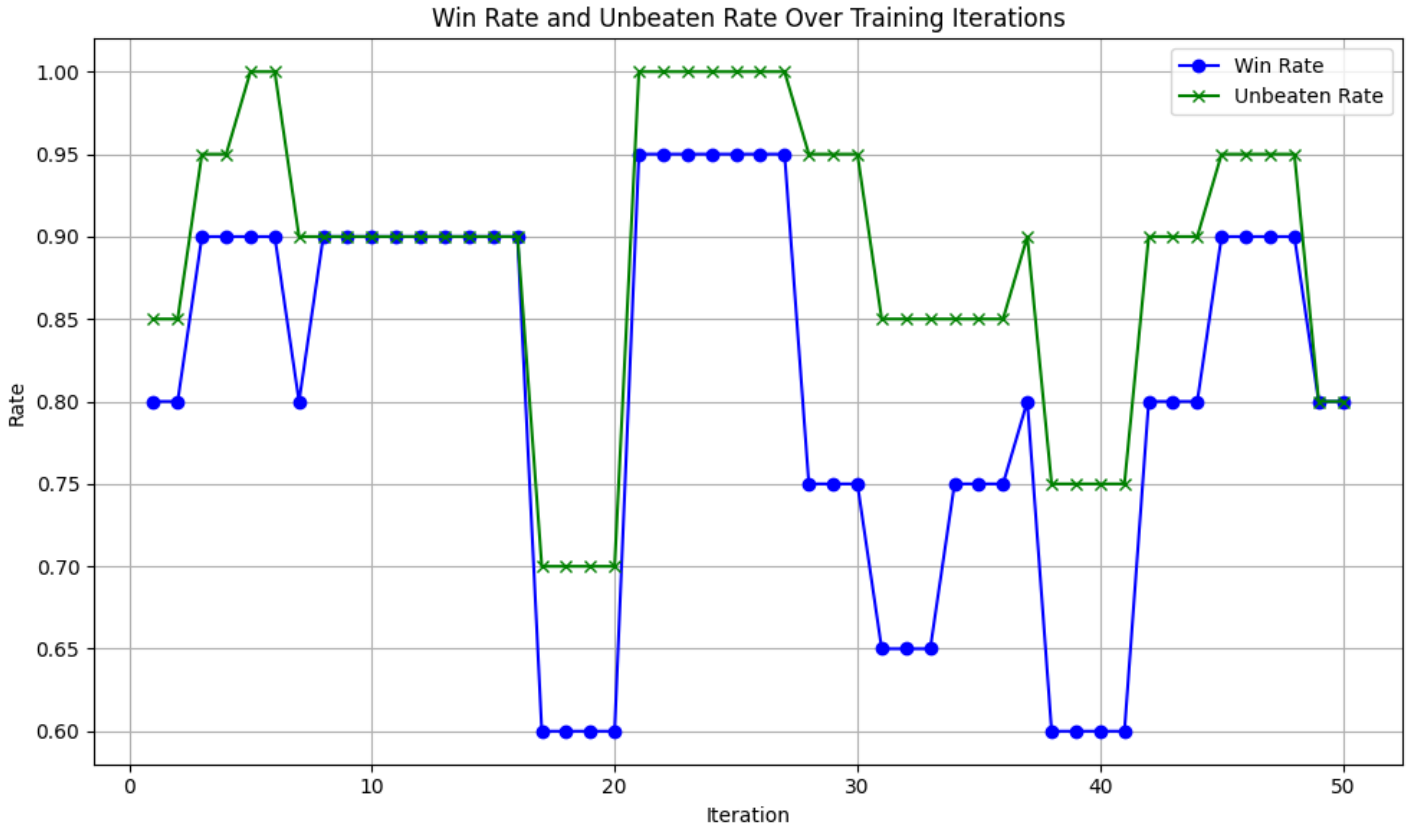
较小的n_search：每次决策时只进行少量的搜索。这意味着决策速度较快，但由于搜索深度有限，模型可能无法发现所有可能的策略，表现可能较差，尤其在较复杂的环境中。

较大的n_search：增加了搜索次数，模型可以探索更多的可能路径，通常能获得更精确的决策。更多的搜索可以帮助模型更好地评估不同动作的潜在结果，从而提高胜率。然而，过多的搜索次数会显著增加计算成本，导致训练和推理的时间变长。

3.6

修改了基函数为：

```python
def base_function(X: np.ndarray) -> np.ndarray:
    return np.hstack([X, X**2, X**3])
```



提升局面策略的学习

棋盘局面中，某些策略可能不单纯依赖于单个位置的状态，而是依赖于多个位置的组合、距离或角度等复杂关系。多项式变换能够帮助模型更好地识别这种复杂的非线性策略。

提高模型的拟合能力

在一些经典的棋类游戏中，局面的评估函数往往具有较强的非线性特性（例如，某些棋子之间的协同作用）。通过引入多项式特征，可以帮助模型学习到这些复杂的非线性关系，提高模型对棋盘局面的拟合能力。

可以看到，在修改了基函数之后，模型的胜率和不败率都有了提高，整体上来看胜率在80%浮动，不败率在90%浮动，模型能力变强了。